

CHAPTER 15

# Software Engineering Economics

ACRONYMS

|      |                                   |
|------|-----------------------------------|
| IRR  | Internal Rate of Return           |
| MARR | Minimum Acceptable Rate of Return |
| SDLC | Software Development Life Cycle   |
| SPLC | Software Product Life Cycle       |
| ROI  | Return on Investment              |
| SEI  | Software Engineering Institute    |
| TCO  | Total Cost of Ownership           |

INTRODUCTION

Software is ubiquitous and has become essential for many organizations. It serves organizations in the following ways:

- as a lever to reach an organization’s business or strategic goals;
- as a catalyst of organizational know-how to improve value.

Both aspects lead directly to critical software engineering demands:

- increased productivity
- reduced rework
- reduced development time
- shorter maintenance turnaround
- long-term sustainability
- innovation
- competitiveness
- alignment with organizational goals

Software engineering economics helps software engineers work in ways that satisfy these critical demands. The Introduction to *SWEBOK Guide* explains that engineering economics is a

key element of all recognized engineering disciplines. Economics is the science of choice, not the science of money. Engineering economics is the applied microeconomics branch of economics. It asks the fundamental question, “Is it in the best interest of this enterprise to invest its limited resources in this technical endeavor, or would the same investment produce a higher return elsewhere?” Paraphrasing the definition in [1], engineering is “finding the balance between what is technically feasible and what is economically acceptable.”

Software engineering must be *value-based*. Neutral — or worse, negative — value from an organization’s investment in software is not sustainable. Software engineering economics aligns software technical decisions with the organization’s business goals.

“The organization” will at least include the organization where the software engineer is employed. However, when the software engineer is involved in work for any third party, such as through an external digital transformation contract or other “works for hire” situation, the business goals of that third party are also relevant.

In all types of organizations — for-profit, nonprofit and government — a value-based approach translates into long-term sustainability. In for-profit organizations this means achieving a tangible return on the software investment. In nonprofit organizations, this means achieving the maximum benefit for the least cost.

Software technical decisions, like an organization’s decision to use a preexisting library or to develop its own, may appear easy from a purely technical perspective. However, they can have serious implications for the business viability of a software project as well as the product itself. Most software practitioners wonder

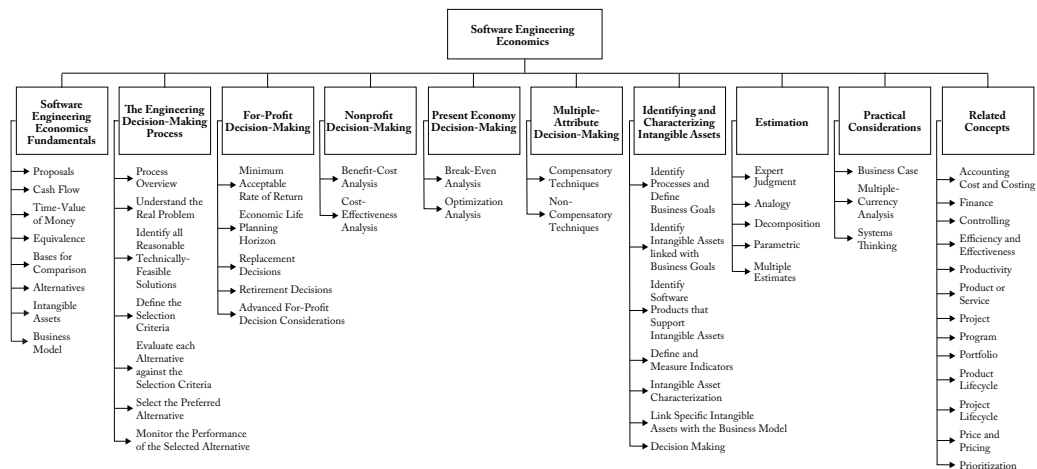


Figure 15.1. Breakdown of Topics for the Software Engineering Economics KA

whether such concerns apply to them. But economic decision-making is fundamental to engineering. Someone who cannot make decisions from both a technical and an economic perspective cannot be considered a true engineer.

Software engineering economics applies to decisions across the entire software product life cycle (SPLC), from the pre-project decision to develop the software to end-of-life decisions for existing software. It also applies to decisions at all levels of technical detail. For example, all following questions involve an economic perspective:

- can a client organization benefit from a digital transformation?
- does a project proposal (a tender) align with a client's business goals?
- should certain software functionality be bought or built?
- should certain requirements be included in scope or not?
- what is the most efficient, cost-effective architecture and design?
- what is an optimal load-balancing strategy for a cloud-based deployment that provides adequate response time to clients without incurring unnecessary operational cost?
- how much risk-based testing is enough?
- is it better to refactor, redevelop or just live with code that has high technical debt?

- is it better to focus maintenance on adding new functionality or on fixing known defects?
- would the value of early delivery of partial functionality gained by using an Agile process outweigh the overhead of rework and continuous testing inherent in iterative approaches?

The Software Engineering Economics knowledge area is directly or indirectly related to all other KAs in this *Guide*.

This KA also takes the position that the more traditional, purely financial view of engineering economics needs to be broadened [2]. Value does not always derive from money alone; value can also derive from “unquantifiables” like corporate citizenship, employee well-being, environmental friendliness, customer loyalty and so on. Therefore, software engineering decisions must also consider relevant unquantifiable criteria.

## BREAKDOWN OF TOPICS FOR SOFTWARE ENGINEERING ECONOMICS

The breakdown of topics for the Software Engineering Economics KA is shown in Figure 15.1.

## 1. Software Engineering Economics Fundamentals

### 1.1. Proposals [3\*, c3pp23-24]

Software engineering decisions begin with the concept of a *proposal* — a single, separate course of action to be considered (e.g., carrying out a particular software development project or not). Another proposal could be to enhance an existing software component; another might be to redevelop that same software from scratch. In deciding what algorithm to use in implementing a certain function, each candidate considered is a proposal. Every proposal represents a binary unit of choice — the software engineer either carries out that proposal or chooses not to. Software engineering economics aims to identify the proposals best aligned with the organization's goals.

### 1.2. Cash Flow [3\*, c3pp24-32]

Engineers must evaluate a proposal from a financial perspective to make a meaningful decision about it. The concepts of *cash flow instance* and *cash flow stream* describe the financial perspective of proposals.

A cash flow instance is a specific amount of money flowing into or out of the organization at a specific time as a direct result of carrying out a proposal. For example, in a proposal to develop and launch product X, the payment for new computers, if needed, would be an example of an outgoing cash flow instance. Money would need to be spent to carry out that proposal. The sales income from product X in the 11th month after market launch would be an example of an incoming cash flow instance. Money would come in because of carrying out the proposal.

A cash flow stream is the set of cash flow instances over time caused by carrying out that proposal. The cash flow stream is that proposal's complete financial view. How much money goes out? When does it go out? How much money comes in? When does it come in? If the cash flow stream for Proposal A is more desirable than the cash flow stream

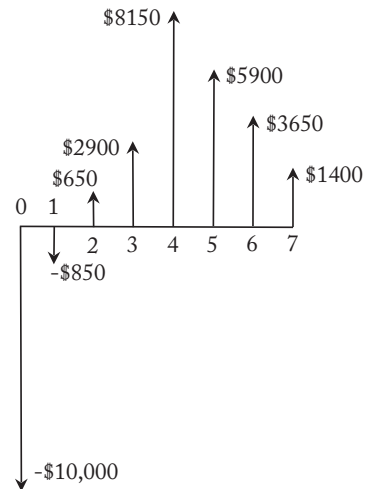


Figure 15.2. A Cash Flow Diagram

for Proposal B, then — all other things being equal — the organization is financially better off carrying out Proposal A than Proposal B. Thus, the cash flow stream is an important element of engineering decision-making.

A *cash flow diagram* is a picture of a cash flow stream. The cash flow diagram quickly summarizes the financial view of a proposal. Figure 15.2 shows an example cash flow diagram.

The cash flow stream is shown in two dimensions. Time runs from left to right and amounts of money run up and down. The horizontal axis is divided into units representing years, months, weeks, etc., as appropriate for the proposal. Each net cash flow instance is drawn at a left-to-right position relative to its timing. The amount of the cash flow instance is shown as an upward or downward arrow. Upward arrows indicate that money is coming in (income), whereas downward arrows indicate that money is spent (expense). The arrow's length is usually proportional to the net amount.

### 1.3. Time-Value of Money [3\*, c5-6]

One of the most fundamental concepts in economics — and therefore, in business decisions — is that money has time-value: Its value changes over time. A specific amount of money right now almost always has a value

different from the same amount at some other time. This concept has been around since the earliest recorded human history and is commonly expressed as *interest*.

#### 1.4. Equivalence [3\*, c7]

Due to the time-value of money, two or more cash flows are equivalent only when they equal the same amount of money at the same time. Therefore, comparing cash flows makes sense only when they are expressed in the same time frame. Then, lack of equivalence between the two cash flows can be determined accurately and can serve as the basis for choice. The proposal with the highest value in the same time frame is the most financially desirable.

#### 1.5. Bases for Comparison [3\*, c8]

A *basis for comparison* is a shared frame of reference for comparing two or more cash flow streams. It uses equivalence to meaningfully compare two or more proposals. Several bases for comparison exist, including the following:

- present worth;
- future worth;
- annual equivalent;
- internal rate of return (IRR);
- discounted payback period.

#### 1.6. Alternatives [3\*, c9]

Often, an organization could carry out more than one proposal if it wanted to. But there might be important relationships between proposals that need to be considered. Maybe Proposal Y can only be carried out if Proposal X is also carried out. Or maybe Proposal P cannot be carried out if Proposal Q is carried out, nor could Q be carried out if P were. Decisions are easier when there are mutually exclusive paths — A, or B, or C, or another project, and no others. This topic explains how to turn any set of proposals, with their interrelationships, into a set of mutually exclusive alternatives. The cash flow stream for any alternative is the sum of the cash flow streams

for all the proposals it contains. The choice can then be made among these alternatives.

One special case is known as the *do-nothing alternative*. Sometimes the best course of action is not to carry out any of the proposals being considered. The do-nothing alternative represents that case. It doesn't mean do nothing at all; it means "do something else, something that's not in this set of choices." The do-nothing alternative should be considered in most, but not all, situations.

#### 1.7. Intangible Assets

*Intangible assets*, also known as *knowledge assets*, are any knowledge that lies in the non-visible side of an organization but affects that organization's financial performance. According to International Valuation Standards (IVS) 210 § 20.1, "an intangible asset is a non-monetary asset that manifests itself by its economic properties. It does not have physical substance but grants rights and economic benefits to its owner" [4].

This can include, but is not limited to, policies, procedures, tools and specifications, as well as organizational culture, experience and know-how.

Knowing the organization's intangible assets helps the software engineer better understand how proposals may affect or be affected by organizational realities. Otherwise, hidden risks and opportunities that could influence proposals' success or failure might not be exposed.

The skills needed to consider intangible assets are the following:

- intangible assets identification and valuation [Skills Framework for the Information Age (SFIA), category Strategy and Architecture, subcategory Business strategy and planning];
- knowledge management [SFIA, category Strategy and Architecture, subcategory Business strategy and planning].

Identifying and characterizing intangible assets are discussed in more detail later in this KA.

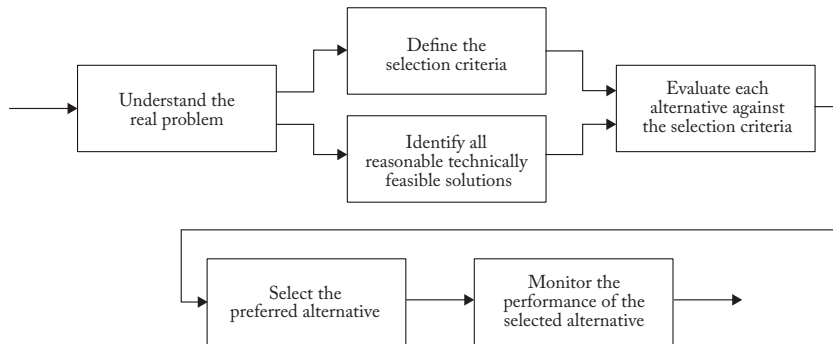


Figure 15.3. The Engineering Decision-Making Process

### 1.8. Business Model

Peter Drucker, a founder of modern management, defines a good business model as one that answers these questions [5]:

- Who is the customer?
- What does the customer value?
- How do we make money?
- What is the underlying economic logic that explains how we can deliver value to customers at an appropriate cost?

Understanding the organization's business model — as well as its intangible assets — helps the software engineer better understand how proposals may affect or be affected by organizational realities. Analyzing the business model can help the software engineer identify hidden risks and opportunities that could influence a proposal's success or failure [6].

## 2. The Engineering Decision-Making Process

### 2.1. Process Overview [3\*, c4pp35-36]

Figure 15.3 provides an overview of the engineering decision-making process.

The process is shown as stepwise and sequential; however, it can be more fluid in practice. Steps can be done iteratively, can overlap and can even occur in different sequences. Just be sure not to skip any step or execute it poorly.

When the consequences of a wrong decision are significant, such as a go/no-go decision for a large project, more time, effort and care should be spent in this process. All steps should be completed thoroughly and carefully. ISO 12207 [7] and ISO 15288 [8] recommend two additional early activities, which can be important in high-consequence decisions:

- define the decision management strategy — this strategy might specify roles, responsibilities, procedures and tools;
- identify relevant stakeholders, which might include appropriate subject matter experts.

When the consequences of a wrong decision are small, such as the consequences of selecting a minor algorithm or data structure, less time, effort and care can be spent, but the same general process is followed. Each step is discussed in more detail below.

### 2.2. Understand the Real Problem [3\*, c4pp37-39]

The best solution to a problem can come only from thoroughly understanding the real problem to be solved. This step's key aspects include the use of an interrogative technique such as the "5 Whys" technique and a consideration of the broader context surrounding the problem. The Empathize stage in Design Thinking [9] (to consider intangible assets) and looking closely at the organization's

business model are examples of considering that broader context.

### *2.3. Identify All Reasonable Technically Feasible Solutions* [3\*, c4pp40-41]

The goal of engineering decision-making is to find the best solution. However, the best solution must first be identified as a candidate before it can be selected as the best. If the best solution is not among the set of solutions being considered, it cannot be selected. The importance of creative thinking in this step cannot be overstated when the consequences of a wrong decision are high.

For some potential solutions, or candidates, prototyping is a useful way to verify technical feasibility. Peer review can also help verify technical feasibility and possibly spur the identification of even more candidates.

On the one hand, adding candidates increases the probability that the best one is in the set. On the other hand, each adds cost to the decision-making process. Software engineers must use their best judgment in deciding when they have enough candidates. These candidates are the “proposals” as defined in the Fundamentals topic, Section 1.

### *2.4. Define the Selection Criteria* [3\*, c4pp39-40, c26pp441-442]

Engineering decisions almost always consider the financial perspective. However, other decision criteria can also be relevant; when this is the case, the decision is a multiple-attribute decision. For example, an environmentally conscious organization may choose a less economical solution if it is more eco-friendly. In many cases, the greater the consequences of a wrong decision, the more selection criteria need to be considered.

As much as possible, each criterion should be expressed objectively. Ideally, those objective terms will be expressed as a monetary value — but not necessarily. What is the “value” of a clean river? It might not make sense to value a river by multiplying the price per kilogram of fish by an estimate of the number of fish in the

river. Decision criteria that can’t be expressed objectively are called “unquantifiables,” “irreducibles” or “intangibles.”

Defining the decision criteria can be a subjective task. Too many criteria could make the analysis unwieldy. On the other hand, too few criteria might not differentiate well between proposals and could thus lead to a suboptimal choice. The potential for a better decision provided by including more criteria must be balanced against the extra effort required to analyze the criteria.

To the extent that money is a selection criterion, the context of the decision will constrain the decision-maker to a for-profit, nonprofit or present economy decision analysis, as explained in topics 3, 4 and 5, later in this KA.

### *2.5. Evaluate Each Alternative Against the Selection Criteria* [3\*, c4pp41-42]

Each alternative is evaluated against each selection criterion. When a selection criterion involves money, each alternative must be judged from the same viewpoint. Use the same basis for comparison (present worth, future worth, IRR, etc., in for-profit decisions; benefit-cost ratio or cost-effectiveness in nonprofit decisions, etc.), the same planning horizon, and consider the same kinds of costs and incomes. An example decision might be buying and adapting an off-the-shelf software product versus building a custom application from scratch. Considering costs for a longer time frame for one proposal than for the other will make the one using the shorter time frame seem like the better choice even though it might not be better over the same time frame.

### *2.6. Select the Preferred Alternative* [3\*, c4p42, c26pp447-458]

If the only selection criterion is money, the alternative with the highest present worth, future worth, etc., will be chosen. When there are multiple criteria, a variety of techniques can be used to evaluate the criteria together. Multiple-attribute decision-making is detailed later in this KA.



Engineering decisions are based on estimates (discussed later in this KA). The accuracy of an estimate is limited in theory and in practice, and the degree of inaccuracy depends on the specifics of the situation [3\*, c21pp344-356]. If the degree of inaccuracy is high enough, that inaccuracy could change the resulting decision. The following techniques [3\*, c23] can help engineers address these situations:

- consider ranges of estimates;
- perform a sensitivity analysis;
- delay final decisions.

In addition, two categories of techniques address multiple potential outcomes from a decision:

- decision-making-under-risk techniques [3\*, c24] are used when probabilities can be assigned to the different potential outcomes. Specific techniques include expected value decision-making, expectation variance and decision-making, Monte Carlo analysis, decision trees, and the expected value of perfect information;
- decision-making-under-uncertainty techniques [3\*, c25] are used when probabilities cannot be assigned to the different potential outcomes. Specific techniques include the Laplace Rule, the Maximin Rule, the Maximax Rule, the Hurwicz Rule and the Minimax Regret Rule.

High-consequence decisions may benefit from formally recording the selected alternative and the justification for why that alternative was selected.

### *2.7. Monitor the Performance of the Selected Alternative* [3\*, c4pp42-43]

Because estimation is a fundamental element of engineering decision-making, the quality of the decision depends on the quality of the estimates. Bad estimates can easily lead to bad decisions. The software engineer needs to “close the loop” on estimates by comparing them to the actual outcomes. Otherwise, no one will ever

know if the estimates were good [3\*, c21pp356-358]. This also helps improve estimation over time. Understanding what drives differences between estimates and actual outcomes helps engineers refine estimation techniques to produce more accurate estimates in the future.

## **3. For-Profit Decision-Making**

For-profit decision techniques apply when the organization’s goal is profit — which is the case in most companies.

Figure 15.4 shows the process for identifying the financially best alternative out of a set of proposals. Arranging alternatives in order of increasing initial investment and then selecting strictly better candidates means that, all other considerations being equal, the alternative with the smaller initial investment will be chosen. The “Is the next candidate strictly better?” decision is made in terms of the appropriate basis for comparison: present worth, future worth, IRR, etc.

### *3.1. Minimum Acceptable Rate of Return* [3\*, c10pp141-143]

The *minimum acceptable rate of return* (MARR) is the lowest IRR the organization would consider a good investment. Generally, it would not be wise to invest in an activity with a return of 10% when another activity returns 20%. The MARR is a statement that the organization is confident it can achieve at least that rate of return. The MARR represents the organization’s opportunity cost for investments. By investing in some alternative, the organization explicitly decides not to invest that same money somewhere else. If the organization is already confident it can achieve a known rate of return, alternatives should be chosen only if their rate of return is at least that high. A simple way to account for that opportunity cost is to use the MARR as the interest rate in the basis for comparison.

### *3.2. Economic Life* [3\*, c11pp160-164]

When an organization invests in a particular alternative, money is tied up in that

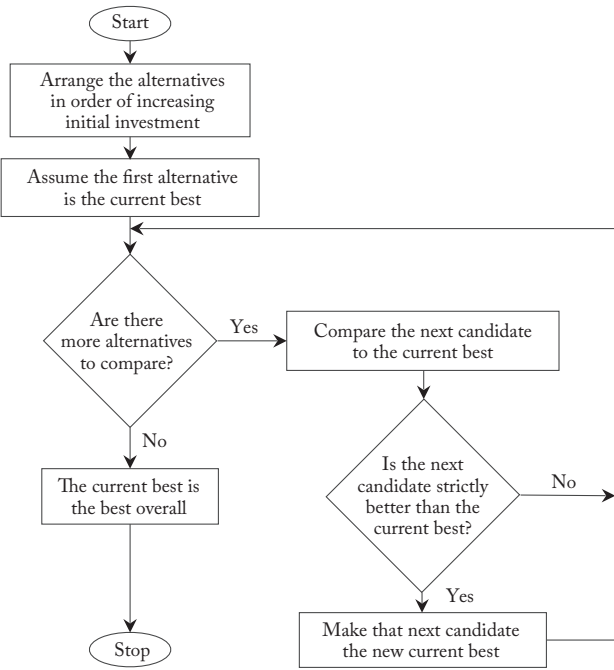


Figure 15.4. The For-Profit Decision-Making Process

alternative — so-called *frozen assets*. The economic impact of frozen assets typically starts high and decreases over time. On the other hand, operating and maintenance costs tend to start low and increase over time. An alternative’s total cost is the sum of those two costs. At first, frozen asset costs dominate; later, operating and maintenance costs dominate. At some point, the sum of the two costs is minimized; this is the *economic life* or *minimum cost lifetime*.

3.3. Planning Horizon [3\*, c11]

To properly compare a proposal with a four-year life to a proposal with a six-year life, the economic effects of either cutting the six-year proposal by two years or investing the profits from the four-year proposal for another two years need to be addressed. The *planning horizon*, sometimes known as the *study period*, is the consistent time frame over which all proposals in the same decision are considered. Aspects such as economic life and the time

frame over which reasonable estimates can be made need to be factored into establishing a planning horizon. Once the planning horizon is established, several techniques are available for putting proposals with different life spans into that planning horizon.

3.4. Replacement Decisions [3\*, c12pp171-178] [8\*, c9]

A replacement decision happens when an organization already has a particular asset and is considering replacing it with a different asset (e.g., deciding between maintaining and supporting legacy software or redeveloping it from the ground up). Replacement decisions use the same for-profit decision process, but there are two additional important considerations: sunk cost and salvage value. Replacement does not necessarily need to involve an entire asset. To the extent that an asset can be replaced in smaller increments, the decision-maker can consider incremental replacement options among the various economic alternatives.



### 3.5. Retirement Decisions

[3\*, c12pp178-181] [8\*, c9]

Retirement decisions are about getting out of an activity altogether, such as when a software company considers not selling a software product anymore or a hardware manufacturer considers not building and selling a particular computer model any longer. Retirement decisions can be preplanned or happen spontaneously (e.g., when performance targets are not achieved). Retirement decisions can be influenced by lock-in factors such as technology dependency and high exit costs.

### 3.6. Advanced For-Profit Decision Considerations

[3\*, c13-17]

The above concepts and techniques are often sufficient to make a good for-profit decision. However, particularly when the consequences of a wrong decision are high, additional considerations may need to be factored into the decision analysis, including the following:

- inflation or deflation;
- depreciation;
- income taxes.

## 4. Nonprofit Decision-Making

The for-profit decision techniques don't apply when the organization's goal isn't profit — which is the case in government and nonprofit organizations. These organizations have a different goal, so different decision techniques are needed. The two techniques are benefit-cost analysis and cost-effectiveness analysis (discussed below).

### 4.1. Benefit-Cost Analysis [3\*, c18pp303-311]

Benefit-cost analysis is one of the most widely used methods for evaluating proposals in nonprofit organizations. A proposal's financial benefits are divided by its costs. Any proposal with a benefit-cost ratio of less than 1.0 can usually be rejected without further analysis because it would cost more

than it would benefit the organization. Additional considerations are necessary when two or more proposals are considered at the same time.

### 4.2. Cost-Effectiveness Analysis

[3\*, c18pp311-314]

Cost-effectiveness analysis shares much of the philosophy and methodology of benefit-cost analysis. There are two versions of cost-effectiveness analysis. The fixed-cost version seeks to maximize benefit given a fixed upper bound on cost. The fixed-effectiveness version seeks to minimize the cost to achieve a fixed goal.

## 5. Present Economy Decision-Making

This subset of engineering decision-making is called *present economy* because it does not involve the time-value of money (future economy). The two forms of present economy decisions are presented below.

### 5.1. Break-Even Analysis

[3\*, c19]

Given functions describing the costs of two or more proposals, break-even analysis helps engineers choose between them by identifying points where those cost functions are equal. Below a break-even point, one proposal is preferred, and above that point, the other is preferred. For example, consider a choice between two cloud service providers. One provider has a lower fixed cost per month with a higher incremental fee for use, whereas the other has a higher fixed cost per month with a lower incremental fee for use. Break-even analysis identifies the use level where the costs are the same. The organization's expected use level can be compared to the break-even point to identify the lower-cost provider.

### 5.2. Optimization Analysis

[3\*, c20]

Optimization analysis studies one or more cost functions over a range of values to find the

point where overall cost is lowest. Software's classic space-time trade-off is an example of optimization; an algorithm that runs faster often uses more memory. Optimization balances the value of faster run time against the cost of the additional memory.

## 6. Multiple-Attribute Decision-Making [3\*, c26]

Most topics presented in this KA so far have discussed decisions based on a single criterion — money. The alternative with the best present worth, the best incremental IRR, the best incremental benefit-cost ratio, etc., is the one selected. Aside from technical feasibility, money is usually the most important decision criterion, but it's certainly not always the only one. Often, other criteria, other "attributes," need to be considered that can't be cast in terms of money. Multiple-attribute decision-making techniques allow nonmonetary criteria to be factored into the decision.

A variety of techniques can be used to address multiple criteria, including nonmonetary criteria. These techniques fall into two categories.

### 6.1. Compensatory Techniques [3\*, c26pp449-458]

Also called *single-dimensioned techniques*, the techniques in this category collapse all criteria into a single figure of merit. This category is called *compensatory* because, for any given alternative, a lower score in one criterion can be compensated by — traded off against — a higher score in other criteria. Compensatory techniques include nondimensional scaling, additive weighting and analytic hierarchy process (AHP).

Gilb's Impact Estimation [11] and the Software Engineering Institute's (SEI) Architectural Tradeoff Analysis Method (ATAM) [12] are examples of compensatory multiple-attribute decision-making techniques focused on identifying the best software design.

### 6.2. Non-Compensatory Techniques [3\*, c26pp447-449]

Also called *fully dimensioned techniques*, the techniques in this category do not allow trade-offs among the criteria. Each criterion is treated as a separate entity in the selection process. Non-compensatory techniques include dominance, satisficing and lexicography.

## 7. Identifying and Characterizing Intangible Assets

The intangible side of an organization is the valuable knowledge residing within it. This includes employees' knowledge about processes, structures, procedures, etc. (tacit, or implicit, knowledge), as well as institutional knowledge recorded in various organizational resources (explicit knowledge). These assets are usually hidden, the way most of an iceberg is underwater. The intangible assets must be explicitly considered in many decisions, particularly when the consequences of a wrong decision are high for the client, no matter if the client is internal or external to the organization for which the software project is being done.

If these assets are not adequately considered, software engineers risk developing a software solution that does not fit the client organization. Only when the intangible assets are explicitly considered will the risk of a poorly fitting software solution be minimized. The Strategic Intangible Process Assets Characterization (SIPAC) method [13] has been used to good effect to accomplish this. SIPAC steps are outlined in the following subsections.

### 7.1. Identify Processes and Define Business Goals

Start by understanding the organization's business processes and business goals. If the organization has well-documented processes, these should be used; otherwise, a deliberate survey will be necessary.

Business goals can include, but are not limited to, the following:

1. maintaining growth and continuity of the organization;
2. meeting financial objectives;
3. meeting responsibility to employees;
4. meeting responsibility to society;
5. managing market position.

### *7.2. Identify Intangible Assets Linked with Business Goals*

The next step is to comprehensively identify the intangible assets. Common examples are policies, documented business processes, checklists, lessons learned, templates, standards, procedures, plans and training materials. Organizations develop or acquire these assets to meet their business goals. The assets represent investments that provide business value. One effective approach to identifying an organization's intangible assets is to start with a taxonomy such as described in the following reference [14]. The goal is to identify as many intangible assets as possible that serve as a lever to achieve the business goals identified in the previous step. In practice, this can be an iterative process where reviewing the already-identified assets reveals the existence of others. A practical way to do it is by focusing iteratively on the 11 generic intangible assets (GIAs) described in [6].

Possible GIAs represent all potential parts of any business that can be involved in a digital transformation. Focusing on one or more of them allows the software engineer to better understand and frame the project's impact. Focusing iteratively on the 11 GIAs, the software engineer will select the type of GIA to be considered and, with this, elicit the specific intangible assets associated with each GIA.

In addition to identifying specific intangible assets, a qualitative relative "importance" must be added to each one as it is identified. The importance is a value between 1 and 5 (1 for lower importance and 5 for higher importance), depending on how well the asset supports achieving the business objectives. The intangible assets with the highest importance are likely the most suitable target for the client organization.

### *7.3. Identify Software Products That Support Intangible Assets*

Software products that support specific intangible assets will be part of the digital transformation proposal to be presented to the client to help them decide what digital transformation to implement.

To identify products related to specific intangible assets, the software engineer can choose from the following:

- discovering them all at a single time by using the methodology of Osterwalder [13], which promotes innovation by generating a value map with the stakeholders' emerging needs, mapping the products to the specific intangible assets;
- listing them if they are known and then mapping them to specific intangible assets;
- iteratively working with the individual intangible assets by (1) selecting a specific intangible asset and (2) identifying the products, continuing until all specific intangible assets have been analyzed.

A single product can support more than one specific intangible asset, and each specific intangible asset can be supported by many products.

### *7.4. Define and Measure Indicators*

This step defines and measures the indicators that will be used to characterize how the intangible assets (through the software products that support those intangible assets) help meet business goals through describing, implementing or improving the identified products. Quality indicators assess specific intangible asset characteristics or features. Impact indicators assess how much the specific intangible assets contribute to processes or business goals.

Indicators must be normalized and standardized to operate correctly.

### *7.5. Intangible Asset Characterization*

Based on the information gathered, the software engineer determines the value of the

identified specific intangible assets based on their quality and impact. Specific intangible assets may be characterized in terms of their impact on business goals and their quality as organizational assets. There are three important characterization cases:

- case 1: specific intangible assets with both impact and quality indicators (Warning, Replaceable, Evolving or Stable);
- case 2: specific intangible assets with only quality indicators (Acceptable Quality or Unacceptable Quality);
- case 3: specific intangible assets with only impact indicators (Acceptable Impact or Unacceptable Impact).

The three characterization cases are shown in Figure 15.5. The quadrants represent the “states” constituting different levels of characterization. The lines separating the quadrants are thresholds of impact and quality that define the point at which the impact or quality of a specific intangible asset may be considered acceptable or not for each organization. These thresholds are established for every client organization and specify what level of organizational performance, quality, and impact they will demand from their knowledge/intangible assets. Thresholds are used to determine when quality and/or impact are acceptable or unacceptable. Let’s look at an example of how to interpret Qval and Ival (both Qval and Ival will be explained in the following sections). Assuming, for example, that we are analyzing the status of an intangible asset with both quality and impact indicators, and that Qval is below the quality threshold and Ival is below the impact threshold. In these circumstances we would say that the status of the intangible asset is “warning” as can be seen in Figure 15.5.

The characterization uses information from standardized-normalized indicators to assess the identified intangible assets. This assessment generates a descriptive value that will determine the asset’s general state of health from a quantitative perspective.

### Quality quantitative assessment

The quality valuation considers only the indicators of the type *quality* of an intangible asset and calculates a general valuation of it. To evaluate the subset of quality indicators, given a set of *q* quality indicators for an intangible asset *n*, the valuation of the quality is given according to Equation 1.

$$Q_{Val}^n = \frac{\sum_{i=1}^q X_i^n}{q}$$

*Equation 1. Quality Assessment for a Knowledge Asset*

Where  $X_i^n$  is each of the *q* normalized indicators of quality that the intangible asset *n* has.

Above, *Qval* is the average of the normalized values of the quality indicators of a corresponding specific intangible asset.

### Impact quantitative assessment

An intangible asset’s impact valuation is an assessment that considers only the normalized indicators that are classified as “impact” indicators. To evaluate the subset of impact indicators, given a set of *p* normalized impact indicators for an intangible asset *n*, the valuation is given as stated in Equation 2:

$$I_{Val}^n = \frac{\sum_{i=1}^p Z_i^n}{p}$$

*Equation 2. Impact Assessment for a Specific Intangible Asset*

Where  $Z_i^n$  is each of the *p* normalized indicators of impact that the knowledge asset *n* has.

Where *Ival* is the average of the normalized values of the impact indicators of a corresponding knowledge asset.

### Linear value calculation

Finally, the linear value of an intangible asset characterization is given by the quality and

impact valuations ( $Qval$  and  $Ival$ ), following these rules, assuming that both quality and impact are equally important, so  $KAval$  (the valuation of the intangible asset) is given by:

If  $\exists Qval \cap \exists Ival$ , then

$$KAval = \frac{Qval + Ival}{2}$$

If  $\exists Qval \cap \nexists Ival$ , then

$$KAval = Qval$$

If  $\nexists Qval \cap \exists Ival$ , then

$$KAval = Ival$$

This linear value represents an intangible asset's general state based on the state of its indicators. It uses the algebraic average of the standardized and normalized indicators to represent the assets' general state on a scale  $[-1, 1]$  and based on the corresponding interpretation thresholds. If no threshold is explicitly mentioned, the linear value is interpreted as follows, if the value is 0, then the intangible asset is on the target, if the value is 1, it means that the intangible asset is 100% over the target and if the value is -1 then the intangible asset is -100% under the target.

#### 7.6. Link Specific Intangible Assets with the Business Model

Visualizing the client business model, enriched with the intangible assets status allocated into that model, gives organizational leadership a clear understanding of the important relationships among proposed software solutions, intangible assets, the business model and the business goals. The software engineer can clearly show which proposed solution generates the most value for the business. An example is shown in [6].

#### 7.7. Decision-Making

The next step in the decision-making process is to prioritize and choose the software products that interest the client organization most. There is no simple rule; several criteria must be considered:

- the intangible asset's impact on business goals (defined in previous steps);
- the characterization reached (defined in previous steps);
- the impact of intangibles assets status on the competitors of the organization under improvement;
- the intangible asset's impact on the business model;
- cost to implement the products;
- time to implement the products;
- complexity of the products.

All criteria must be considered to decide what software products should be developed for the client organization, making this a multiple-attribute decision. (See 6, Multiple-Attribute Decision-Making.)

Upon considering all relevant criteria, the organization can see the risks of implementing a software solution to automate processes that are either not very valuable or not in good shape. Instead, the software engineer can offer, in a transparent way, proposals that better satisfy the organization's business needs.

This approach can be useful whenever an engineering decision needs to be made, but it is particularly critical in the pre-project stage when there is a need to present the client organization with proposals that are best for business value.

## 8. Estimation

[3\*, c21-26]

An estimate analytically predicts some quantity, like a project's size, cost or schedule. Many other quantities are also estimated in software engineering, such as the average number of active client sessions a cloud service needs to support, the number of times a function will be called during execution of a section of code, or the number of delivered defects in a software product.

Software engineers do not estimate purely for the sake of estimating. Software engineers estimate to make decisions when critical quantities are unknown. For example, a decision to buy a functionality or build it within

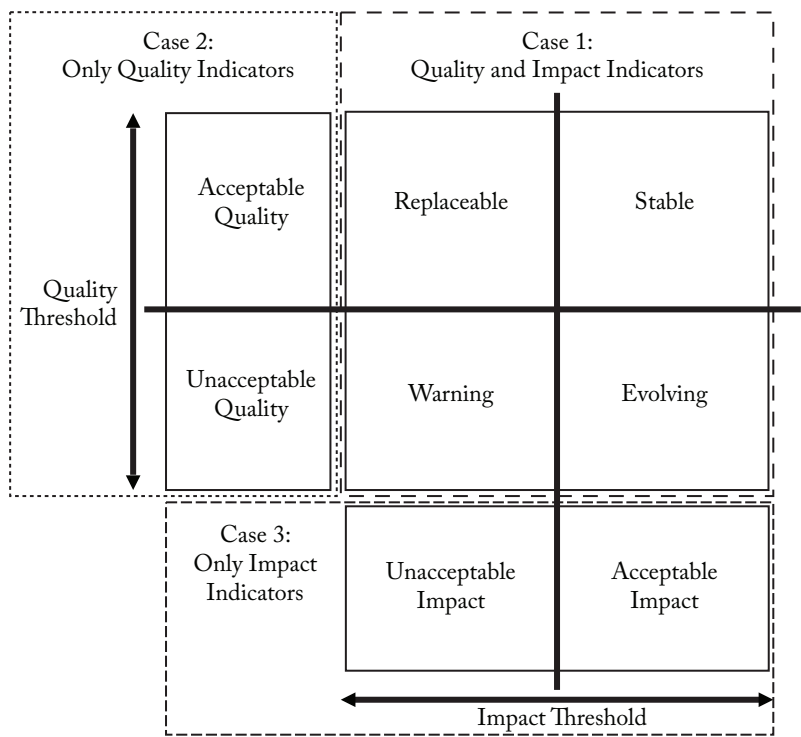


Figure 15.5. Extended Characterization of Specific Intangible Assets

the organization would certainly be based on the cost of building it. But the actual cost of building it cannot be precisely known until the organization does build it. Key information needed to make engineering decisions is usually not known when those decisions need to be made. Instead, decisions are based on estimates. Behind every estimate is one or more decisions.

Given that estimates are predictions, there is a nonzero probability that the actual outcome will differ from the estimate. All estimates are inherently uncertain. Sometimes, the uncertainty is large, and sometimes it is small. But it is always there. Fortunately, estimates need not be perfect; they need only to be good enough to lead the decision-maker to make the right decision.

The Software Engineering Code of Ethics and Professional Practice [16] states, “3.09. Ensure realistic quantitative estimates of cost, scheduling, personnel, quality and outcomes

on any project on which they work or propose to work and provide an uncertainty assessment of these estimates” (underlining added for emphasis). (See [3\*, c21pp358-361].)

Estimation is covered extensively in [17], [18] and [3\*]. Several general techniques exist, and each is overviewed here. All specific estimation techniques use one or a combination of these general techniques.

8.1. Expert Judgment [3\*, c22pp367-369]

In expert judgment estimation, the estimate is based purely on the estimator’s professional opinion. It is the simplest technique and is always available, and it is particularly useful when the other techniques aren’t available. The downside is that this technique produces the least accurate estimates. Multiple expert judgment estimates can be fed into group estimation processes like Wide Band Delphi and Planning Poker to produce more accurate estimates.



### 8.2. *Analogy* [3\*, c22pp369-371]

Estimation by analogy assumes that if the thing estimated is similar to something already known, then the estimate for the new thing can be based on the actual result for the similar thing, with allowances for relevant differences. The steps in estimation by analogy are as follows:

1. Understand the thing to be estimated.
2. Find a suitable analogy for which actual results are known.
3. List differences between the thing being estimated and the analogy that could significantly affect the outcome.
4. Estimate the magnitude of each identified difference.
5. Build the estimate from the analogy's actual result and adjustments for the identified differences.

Estimation by analogy produces more accurate results than expert judgment, and it is still relatively quick and easy. On the other hand, an appropriate analogy for which accurate results are known must be available for this approach to work.

### 8.3. *Decomposition* [3\*, c22pp371-374]

Sometimes called *bottom-up estimation*, the steps in estimation by decomposition are:

1. Break the thing to be estimated into successively smaller pieces until the smallest pieces can be reasonably estimated.
2. Estimate those smallest pieces.
3. Add up the estimates for the smallest pieces to build the estimate for the whole.
4. If the estimates for the smallest pieces don't include allowances for significant cross-cutting factors, then find a way to address those factors. For example, when estimating a software project from its design, the estimates for the design elements may not include allowances for requirements work, integration work, testing work and user documentation work.

Estimation by decomposition assumes that overestimates of lowest-level pieces will cancel out corresponding underestimates of other pieces and lead to a more accurate estimate of the whole. The primary disadvantages are the following:

- it can be a lot more work than any other technique;
- if the bottom-level estimates are biased either high or low, the canceling effect doesn't happen.

### 8.4. *Parametric* [3\*, c22pp374-377]

Also called *estimation by statistical methods*, parametric estimation takes advantage of a known, mathematical relationship between the thing being estimated and one or more observable factors about that thing, like calculating the cost to build a building as a function of its floor space. The estimation model is an equation: First, count the observable factors, and then plug them into the equation to get the resulting estimate.

Parametric estimates are typically the most accurate, the most defensible and the easiest to use, provided the equation has been developed and validated. The disadvantage is that developing and validating such an equation requires an adequate base of accurate historical data along with some nontrivial mathematics and statistics.

### 8.5. *Multiple Estimates* [3\*, c22pp377-379]

When the consequences of a wrong decision are small, it can be acceptable to base the decision on a single estimate from a single estimator using a single estimation technique. However, when the consequences of a wrong decision are significant, investing extra effort in developing more than one estimate can be worthwhile.

To use this approach, engineers estimate the same thing using different techniques, possibly by different estimators. Then, they look for convergence or divergence among those multiple estimates. Convergence suggests

the individual estimates are probably accurate, and any of them could be used to make the decision. Divergence suggests that one or more important factors might have been overlooked. Finding the factors that caused the divergence and reestimating to produce converging results often lead to a better estimate and thus a better decision.

## 9. Practical Considerations

### 9.1. Business Case

The business case is the consolidated, documented information summarizing and explaining a recommended business decision from different perspectives (cost, benefit, risk and so on) for a decision-maker and other relevant stakeholders. It's used to assess a product's potential value, which can be used as a basis for an investment decision.

### 9.2. Multiple-Currency Analysis

When a decision analysis involves cross-border finances, currency exchange rate variations may need to be considered. This is often done using historical data.

### 9.3. Systems Thinking

The ecosystem in which software engineers develop their professional life is complex. To understand the whole picture around a client organization and form a holistic view of the scenarios they analyze, software engineers can use systems thinking methodologies. This approach helps the software engineer create a complete set of possible scenarios in which the software to be provided could be useful and, with this information, explain to the client how the software solution can be a value provider for the organization. Sources for system thinking methodologies are [19] and [20]. A way to connect systems thinking methodologies with the development of a business model to understand the pillars of the client organization can be reached in [21].

## 10. Related Concepts

This topic includes concepts the software engineer may want to bear in mind.

### 10.1. Accounting [3\*, c15pp234-245]

Accounting is part of finance. It allows people whose money is used to run an organization to know the results of their investment: Did they get the profit they were expecting? In for-profit organizations, this relates to the tangible return on investment (ROI), while in nonprofit and governmental organizations, as well as for-profit organizations, it translates into sustainably staying in business. Accounting's primary role is to measure the organization's actual financial performance and to communicate financial information about a business entity to stakeholders, such as shareholders, financial auditors and investors. Communication generally takes the form of financial statements showing the economic resources to be controlled. The right information — relevant and reliable to the user — must be presented. Information and its timing are partially governed by risk management and governance policies. Accounting systems are also a rich source of historical data for estimating.

Software engineers must be conscious of the software's importance as a driver of business accounts in the digital era.

### 10.2. Cost and Costing [3\*, c15pp245-259]

A cost is the money used to produce something and, hence, is no longer available for use. In economics, a cost is an alternative that is given up as a result of a decision.

Sunk cost refers to unrecoverable expenses that have occurred, which can cause emotional hurdles looking forward. From a traditional economics viewpoint, sunk costs should not be considered in decision-making. Opportunity cost is the cost of an alternative that must be forgone to pursue another alternative.

Costing is part of finance and product management. It is the process of determining the cost based on expenses (e.g., production,

software engineering, distribution, rework) and on the target cost to be competitive and successful in a market. The target cost can be below the actual estimated cost. The planning and controlling of these costs (called cost management) is important and should always be included in costing.

An important concept in costing is the total cost of ownership (TCO). This holds true especially for software because there are many not-so-obvious costs related to SPLC activities after initial product development. TCO for a software product is defined as the total cost for acquiring that product, activating it and keeping it running. These costs can be grouped as direct and indirect costs. TCO is an accounting method that is crucial in making sound economic decisions.

### 10.3. Finance

Finance is the branch of economics concerned with allocating, managing, acquiring and investing resources. Finance is an element of every organization, including software engineering organizations.

The field of finance deals with the concepts of time, money, and risk, and how they are interrelated. It also deals with how money is spent and budgeted. Corporate finance is concerned with funding an organization's activities. Generally, this involves balancing risk and profitability while attempting to maximize an organization's wealth and the value of its stock. This applies primarily to for-profit organizations but also to nonprofit organizations. The latter needs finances to ensure sustainability, if not to make a tangible profit. To do this, an organization must:

- identify organizational goals, time horizons, risk factors, tax considerations and financial constraints;
- identify and implement the appropriate business strategy, such as which portfolio and investment decisions to take, how to manage cash flow and where to get the funding;
- measure financial performance, such as

cash flow and ROI, and take corrective actions in case of deviation from objectives and strategy.

Provided that many organizations use software development or acquisition to stay competitive, the software engineer must be conscious of the importance of software to business finances.

### 10.4. Controlling

Controlling is the element of finance and accounting that involves measuring and correcting performance. It ensures that an organization's objectives and plans are accomplished. Controlling cost is a specialized branch of controlling used to detect variances of actual costs from planned costs.

In software engineering, this concept is referred to as processes and products control and evolution. While the organization is seen as an entity with its own goals, and control of the organizational goals is seen as separate, software engineers must consider control of the organization part of their job by ensuring alignment of their software with business goals.

### 10.5. Efficiency and Effectiveness

[10\*, c22pp422-23]

*Economic efficiency* of a process, activity or task is the ratio of resources consumed to resources expected to be consumed. Efficiency means "doing things right." An efficient behavior, like an effective behavior, delivers results and minimizes effort. Factors affecting efficiency in software engineering include product complexity, quality requirements, time pressure, process capability, team distribution, interruptions, feature churn, tools and programming language.

Effectiveness is about having impact. It is the relationship between achieved objectives and defined objectives. *Effectiveness* means "doing the right things." Effectiveness looks only at whether defined objectives are reached — not at how they are reached.

### 10.6. Productivity [10\*, c23pp689]

*Productivity* is the ratio of output to input from an economic perspective. *Output* is the value delivered. *Input* covers all resources (e.g., effort) spent to generate the output. Productivity combines efficiency and effectiveness from a value-oriented perspective. Maximizing productivity is about generating the highest value with the lowest resource consumption.

The Guide to the Project Management Body of Knowledge [23] defines rework as “action taken to bring a defective or nonconforming component into compliance with requirements or specifications.” It is worth noting that most software organizations are unaware that the single largest resource consumer is, in fact, rework. In many software projects the cost of rework is higher than the cost of all other project activities combined. The most effective way to increase productivity can be to simply reduce rework. Reducing software project rework involves proactive quality improvement actions (see Chapter 12, Software Quality KA) that either a) identify defects earlier so those defects can be fixed at lower resource cost, b) reduce the degree of defect cost growth (e.g., intentionally simpler code is easier to modify than complex code so actively managing and controlling code complexity reduces the cost of defect repair), and c) prevent defects in the first place by, for example, using appropriate templates and checklists in development and maintenance.

### 10.7. Product or Service

A *product* is a tangible economic good (or output) created in a process that transforms product factors (or inputs) into an output. A *service* is an intangible resource, like consulting. When sold, a product or service is a deliverable that creates both a value and an experience for its consumers. A product or service can be a combination of systems, solutions and materials delivered internally (e.g., an in-house IT solution) or externally (e.g., a

software application), either as is or as a component for another product (e.g., embedded software).

### 10.8. Project [22\*, c2s2.4]

A *project* is “a temporary endeavor undertaken to create a unique product, service, or result” [24]. In software engineering, different project types are distinguished (e.g., product development, outsourced services, software maintenance, service creation, and so on). During its life cycle, a software product may require many projects. For example, during the product conception phase, a project might be conducted to determine customer need and market requirements; during maintenance, a project might be conducted to produce the next version of a product.

### 10.9. Program

A *program* is “a group of related projects, sub-programs, and program activities managed in a coordinated way to obtain benefits not available from managing them individually” [24]. Programs are often used to identify and manage different deliveries to a single customer or market over a time horizon of several years.

### 10.10. Portfolio

*Portfolios* are “projects, programs, sub-portfolios, and operations managed as a group to achieve strategic objectives” [24]. Portfolios are used to group and then manage simultaneously all assets within a business line or organization. Having an entire portfolio to consider helps ensure that the broader impacts of decisions are considered, such as the decision to allocate resources to a specific project, which means that the same resources will not be available for the other projects in the portfolio.

### 10.11. Product Life Cycle

An SPLC includes all activities needed to define, build, operate, maintain and retire

a software product or service and its variants. The SPLC activities of “operate,” “maintain” and “retire” occur in a much longer time frame than initial software development (the software development life cycle (SDLC)). (See Software Life Cycle Models in the Software Engineering Process KA.) Also, the operate-maintain-retire activities of an SPLC consume more total effort and other resources than the SDLC activities. (See Majority of Maintenance Costs in the Software Maintenance KA.) The value contributed by a software product or associated services can be objectively determined during the “operate and maintain” time frame. Software engineering economics should be concerned with all SPLC activities, including activities that take place after the initial product release.

#### *10.12. Project Life Cycle*

Project life cycle activities typically involve five process groups: Initiating, Planning, Executing, Monitoring and controlling, and Closing [23]. (See the Software Engineering Management KA.) The activities within a software project life cycle are often interleaved, overlapped and iterated in various ways [20\*, c2] [25]. (See the Software Engineering Process KA.) For instance, Agile product development within an SPLC involves multiple iterations that produce increments of deliverable software. An SPLC should include risk management and synchronization with different suppliers (if any) while providing auditable decision-making information (e.g., to comply with product liability needs or governance regulations). The software project life cycle and the SPLC are interrelated; an SPLC may include several SDLCs.

#### *10.13. Price and Pricing* [10\*, c23s23.1]

A *price* is what is paid in exchange for a good or service. Price is a fundamental aspect of

financial modeling and is one of the four Ps of the marketing mix. The other three Ps are product, promotion and place. Price is the only revenue-generating element among the four Ps; the rest are costs.

*Pricing* is an element of finance and marketing. It determines what a company will receive in exchange for its products. Pricing factors include manufacturing cost, market placement, competition, market condition and product quality. Pricing applies prices to products and services based on factors such as fixed amount, quantity break, promotion or sales campaign, specific vendor quote, shipment or invoice date, combination of multiple orders, service offerings, and many others. The consumer’s needs can be converted into demand only if the consumer has the willingness and capacity to buy the product. Thus, pricing is crucial in marketing. Pricing is initially done during the project initiation phase and is a part of the “go” decision-making process.

#### *10.14. Prioritization*

*Prioritization* involves ranking alternatives based on common criteria to deliver the best value. For example, in software engineering projects, software requirements are often prioritized to deliver the most value to the client within the constraints of schedule, budget, resources, and technology, or to allow the team to build the product in increments, where the first increments provide the highest value to the customer. (See Requirements Prioritization in the Software Requirements KA and Software Life Cycle Models in the Software Engineering Process KA.) Prioritizing alternatives is at least implicit in the discussion in 2.6., Select the Preferred Alternative, but is explicit when a compensatory technique is used, as described in 7.6., Multiple-Attribute Decision-Making.

**MATRIX OF TOPICS VS. REFERENCE MATERIAL**

|                                                                      | <b>Tockey 2005<br/>[3*]</b> | <b>Sommerville<br/>2016 [10*]</b> | <b>Fairley 2009<br/>[22*]</b> |
|----------------------------------------------------------------------|-----------------------------|-----------------------------------|-------------------------------|
| <b>1. Software Engineering Economics Fundamentals</b>                |                             |                                   |                               |
| <i>1.1. Proposals</i>                                                | c3pp23-24                   |                                   |                               |
| <i>1.2. Cash Flow</i>                                                | c3pp24-32                   |                                   |                               |
| <i>1.3. Time-Value of Money</i>                                      | c5-6                        |                                   |                               |
| <i>1.4. Equivalence</i>                                              | c7                          |                                   |                               |
| <i>1.5. Bases for Comparison</i>                                     | c8                          |                                   |                               |
| <i>1.6. Alternatives</i>                                             | c9                          |                                   |                               |
| <i>1.7. Intangible Assets</i>                                        |                             |                                   |                               |
| <i>1.8. Business Model</i>                                           |                             |                                   |                               |
| <b>2. The Engineering Decision-Making Process</b>                    |                             |                                   |                               |
| <i>2.1. Process Overview</i>                                         | c4pp35-36                   |                                   |                               |
| <i>2.2. Understand the Real Problem</i>                              | c4pp37-39                   |                                   |                               |
| <i>2.3. Identify All Reasonable Technically Feasible Solutions</i>   | c4pp40-41                   |                                   |                               |
| <i>2.4. Define the Selection Criteria</i>                            | c4pp39-40,<br>c26pp441-442  |                                   |                               |
| <i>2.5. Evaluate Each Alternative Against the Selection Criteria</i> | c4pp41-42                   |                                   |                               |
| <i>2.6. Select the Preferred Alternative</i>                         | c4p42,<br>c26pp447-458      |                                   |                               |
| <i>2.7. Monitor the Performance of the Selected Alternative</i>      | c4pp42-43                   |                                   |                               |
| <b>3. For-Profit Decision-Making</b>                                 |                             |                                   |                               |
| <i>3.1. Minimum Acceptable Rate of Return</i>                        | c10pp141-143                |                                   |                               |
| <i>3.2. Economic Life</i>                                            | c11pp160-164                |                                   |                               |
| <i>3.3. Planning Horizon</i>                                         | c11                         |                                   |                               |
| <i>3.4. Replacement Decisions</i>                                    | c12pp171-178 c9             |                                   |                               |
| <i>3.5. Retirement Decisions</i>                                     | c12pp178-181 c9             |                                   |                               |
| <i>3.6. Advanced For-Profit Decision Considerations</i>              | c13-17                      |                                   |                               |
| <b>4. Nonprofit Decision-Making</b>                                  |                             |                                   |                               |
| <i>4.1. Benefit-Cost Analysis</i>                                    | c18pp303-311                |                                   |                               |
| <i>4.2. Cost-Effectiveness Analysis</i>                              | c18pp311-314                |                                   |                               |
| <b>5. Present Economy Decision-Making</b>                            |                             |                                   |                               |



|                                                                |              |             |        |
|----------------------------------------------------------------|--------------|-------------|--------|
| 5.1. Break-Even Analysis                                       | c19          |             |        |
| 5.2. Optimization Analysis                                     | c20          |             |        |
| <b>6. Multiple-Attribute Decision-Making</b>                   |              |             |        |
| 6.1. Compensatory Techniques                                   | c26pp449-458 |             |        |
| 6.2. Non-Compensatory Techniques                               | c26pp447-449 |             |        |
| <b>7. Identifying and Characterizing Intangible Assets</b>     |              |             |        |
| 7.1. Identify Processes and Define Business Goals              |              |             |        |
| 7.2. Identify Intangible Assets Linked with Business Goals     |              |             |        |
| 7.3. Identify Software Products That Support Intangible Assets |              |             |        |
| 7.4. Define and Measure Indicators                             |              |             |        |
| 7.5. Intangible Asset Characterization                         |              |             |        |
| 7.6. Link Specific Intangible Assets with the Business Model   |              |             |        |
| 7.7. Decision-Making                                           |              |             |        |
| <b>8. Estimation</b>                                           |              |             |        |
| 8.1. Expert Judgment                                           | c22pp367-369 |             |        |
| 8.2. Analogy                                                   | c22pp369-371 |             |        |
| 8.3. Decomposition                                             | c22pp371-374 |             |        |
| 8.4. Parametric                                                | c22pp374-377 |             |        |
| 8.5. Multiple Estimates                                        | c22pp377-379 |             |        |
| <b>9. Practical Considerations</b>                             |              |             |        |
| 9.1. Business Case                                             |              |             |        |
| 9.2. Multiple-Currency Analysis                                |              |             |        |
| 9.3. Systems Thinking                                          |              |             |        |
| <b>10. Related Concepts</b>                                    |              |             |        |
| 10.1. Accounting                                               | c15pp234-245 |             |        |
| 10.2. Cost and Costing                                         | c15pp245-259 |             |        |
| 10.3. Finance                                                  |              |             |        |
| 10.4. Controlling                                              |              |             |        |
| 10.5. Efficiency and Effectiveness                             |              | c22pp422-23 |        |
| 10.6. Productivity                                             |              | c23pp689    |        |
| 10.7. Product or Service                                       |              |             |        |
| 10.8. Project                                                  |              |             | c2s2.4 |

|                           |  |          |  |
|---------------------------|--|----------|--|
| 10.9. Program             |  |          |  |
| 10.10. Portfolio          |  |          |  |
| 10.11. Product Life Cycle |  |          |  |
| 10.12. Project Life Cycle |  |          |  |
| 10.13. Price and Pricing  |  | c23s23.1 |  |
| 10.14. Prioritization     |  |          |  |

FURTHER READINGS

Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)* [24].

The *PMBOK® Guide* provides guidelines for managing individual projects and defines project management-related concepts. It also describes the project management life cycle and its related processes, as well as the project life cycle. It is a globally recognized guide for the project management profession.

Project Management Institute and IEEE Computer Society, *Software Extension to the Guide to the Project Management Body of Knowledge (SWX)* [25].

*SWX* provides adaptations and extensions to the generic practices of project management documented in the *PMBOK® Guide* for managing software projects. The primary contribution of this extension to the *PMBOK® Guide* is its description of processes that are applicable to managing adaptive life cycle software projects.

B.W. Boehm, *Software Engineering Economics* [26].

This book is classic reading on software engineering economics. It provides an overview of business thinking in software engineering. Although the examples and figures are dated, it is still worth reading.

C. Ebert and R. Dumke, *Software Measurement* [27].

This book provides an overview of quantitative methods in software engineering, starting with measurement theory and proceeding to performance management and business decision-making.

D.J. Reifer, *Making the Software Business Case: Improvement by the Numbers* [28].

This book is classic reading on making a business case in software and IT industries. Many useful examples illustrate how the business case is formulated and quantified.

REFERENCES

[1] E. DeGarmo et al., *Engineering Economy*, 9th ed., Prentice Hall, 1993.

[2] P. Rodriguez, C. Urquhart, and E. Mendes. “A Theory of Value for Value-based Feature Selection in Software Engineering,” *IEEE Transactions on Software Engineering*, 1, 2020.

[3\*] S. Tockey, *Return on Software: Maximizing the Return on Your Software Investment*, Addison-Wesley, 2005.

[4] *International Valuation Standards (IVS)*, Norwich: Page Bros, 2019.

- [5] K. Voigt, O. Buliga, and K. Michl, *Business Model Pioneers: How Innovators Successfully Implement New Business Models*, Springer, 2017.
- [6] M.-I. Sanchez-Segura, G.-L. Dugarte-Peña, A. Amescua-Seco, and F. Medina-Domínguez, “Exploring How The Intangible Side of an Organization Impacts its Business Model,” *Kybernetes*, Vol. 50 No. 10, pp. 2790-2822. 2021. <https://doi.org/10.1108/K-05-2020-0302>.
- [7] ISO/IEC/IEEE 12207-2:2020 – Systems and software engineering — Software life cycle processes — Part 2: Relation and mapping between ISO/IEC/IEEE 12207:2017 and ISO/IEC 12207:2008, IEEE, 2020, pp. 1-278.
- [8\*] ISO/IEC/IEEE 15288:2023 – Systems and Software Engineering – System Life Cycle Processes, IEEE.
- [9] T. Brown and B. Katz, *Change by Design: How Design Thinking Transforms Organizations and Inspires Innovation*, Revised and updated ed., Harper Collins, 2019.
- [10\*] I. Sommerville, *Software Engineering*, 10th ed., Addison-Wesley, 2016.
- [11] T. Gilb, *Competitive Engineering: A Handbook for Systems Engineering, Requirements Engineering, and Software Engineering Using Planguage*, Elsevier Butterworth-Heinemann, 2005.
- [12] R. Kazman, M. Klein, and P. Clements, “ATAMSM: Method for Architecture Evaluation,” CMU/SEI-2000-TR-004, Software Engineering Institute, August 2000.
- [13] M.I. Sanchez-Segura, A. Ruiz-Robles, F. Medina-Domínguez, and G.L. Dugarte-Peña. “Strategic Characterization of Process Assets Based on Asset Quality and Business Impact,” *Industrial Management and Data Systems*, 117(8), 1720-1734. <https://doi.org/10.1108/IMDS-10-2016-0422>, 2017.
- [14] M.I. Sanchez-Segura, A. Ruiz Robles, F. Medina-Domínguez. “Uncovering Hidden Process Assets: A Case Study.” *Information Systems Frontiers*, <https://www.springerprofessional.de/en/uncovering-hidden-process-assets-a-case-study/11724394>, 2016.
- [15] A. Osterwalder, Y. Pigneur, G. Bernarda, A. Smith, and T. Papadacos, *Value Proposition Design*, Wiley, 2015.
- [16] D. Gotterbarn, K. Miller, and S. Rogerson, “Software Engineering Code of Ethics,” *Commun. ACM* 40, 11, 110-118, doi: 10.1145/265684.265699, 1997.
- [17] S. McConnell, *Software Estimation: Demystifying the Black Art*, 1st ed., Microsoft Press, 2009.
- [18] R.D. Stutzke, *Estimating Software-Intensive Systems Projects, Products, and Processes*, 1st ed., Addison-Wesley, 2005.
- [19] M. Ben-Eli, *Understanding Systems. Systems Innovation*, [https://netzerocities.app/\\_content/files/knowledge/3148/understanding\\_systems\\_thinking\\_\\_\\_systems\\_modelling\\_the\\_sustainability\\_lab\\_2019\\_\\_1\\_.pdf](https://netzerocities.app/_content/files/knowledge/3148/understanding_systems_thinking___systems_modelling_the_sustainability_lab_2019__1_.pdf), 2019.
- [20] J. Sterman, *Business Dynamics: Systems Thinking and Modeling for a Complex World*, McGraw-Hill, 2000.
- [21] S. Pereira, G. Medina, et al., “System Thinking and Business Model Canvas for Collaborative Business Models Design,” *IFIP Advances in Information and Communication Technology*, Vol. 488, pp. 461-468, 2016.

- [22\*] R.E. Fairley, *Managing and Leading Software Projects*, Wiley-IEEE Computer Society Press, 2009.
- [23] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 7th ed., Project Management Institute, 2021.
- [24] Project Management Institute, *PMI Lexicon of Project Management Term*, 2012.
- [25] Project Management Institute and IEEE Computer Society, *Software Extension to the PMBOK® Guide Fifth Edition*, Project Management Institute, 2013.
- [26] B.W. Boehm, *Software Engineering Economics*, Prentice-Hall, 1981.
- [27] C. Ebert and R. Dumke, *Software Measurement*, Springer, 2007.
- [28] D.J. Reifer, *Making the Software Business Case: Improvement by the Numbers*, Addison-Wesley, 2002.